

```
import random
```

```
employees = ["A", "B", "C", "D"]
```

```
days = ["Mon", "Tue", "Wed", "Thu", "Fri"]
```

```
POP_SIZE = 10
```

```
GENERATIONS = 50
```

```
MUTATION_RATE = 0.1
```

```
# Create a random schedule
```

```
def create_schedule():
```

```
    return [random.choice(employees) for _ in days]
```

```
# Fitness: avoid assigning the same employee
```

```
# too many times
```

```
def fitness(schedule):
```

```
    counts = [schedule.count(e) for e in employees]
```

```
    return 100 - max(counts) * 10
```

```
# Crossover
```

```
def crossover(p1, p2):
```

```
    point = random.randint(1, len(days) - 1)
```

```
    return p1[:point] + p2[point:]
```

```
# Mutation
```

```
def mutate(schedule):
```

```
    if random.random() < MUTATION_RATE:
```

```
i = random.randint(0, len(days) - 1)
schedule[i] = random.choice(employees)
return schedule
```

```
# Initial population
```

```
population = [
    create_schedule()
    for _ in range(POP_SIZE)
]
```

```
# Genetic Algorithm
```

```
for generation in range(GENERATIONS):
```

```
# Selection
```

```
population.sort(
    key=fitness,
    reverse=True
)
```

```
parents = population[:5]
```

```
# Create new population
```

```
new_population = parents.copy()
```

```
while len(new_population) < POP_SIZE:
```

```
    p1 = random.choice(parents)
```

```
    p2 = random.choice(parents)
```

```
child = crossover(p1, p2)
```

```
child = mutate(child)
```

```
new_population.append(child)
```

```
population = new_population
```

```
# Best solution
```

```
best = max(population, key=fitness)
```

```
print("Best Schedule:")
```

```
for day, employee in zip(days, best):
```

```
    print(day, "->", employee)
```

```
print("Fitness:", fitness(best))
```

```
08  key=fitness,
    reverse=True
    )

    parents = population[:5]

    # Create new population
    new_population = parents.copy()

    while len(new_population) < POP_SIZE:

        p1 = random.choice(parents)
        p2 = random.choice(parents)

        child = crossover(p1, p2)
        child = mutate(child)

        new_population.append(child)

    population = new_population

    # Best solution
    best = max(population, key=fitness)

    print("Best Schedule:")

    for day, employee in zip(days, best):
        print(day, "->", employee)

    print("Fitness:", fitness(best))

... Best Schedule:
Mon -> D
Tue -> C
Wed -> C
Thu -> B
Fri -> B
Fitness: 80
```

